

Campus Activity Planner

JavaFX desktop application for browsing campus activities and managing registrations.

Object-Oriented Concepts - Summer Semester 2026

Omar Abdelmegid - Xue Wang - Yousuf Sidiqi

What the Application Does

- Shows campus activities such as workshops, guest lectures, sports and game nights
- Lets students register for an activity
- Tracks confirmed registrations, free places and waiting list entries
- Prevents duplicate participants for the same activity
- Supports different registration types with different fees and priorities



JavaFX GUI

The GUI is implemented in `CampusActivityGUI` .

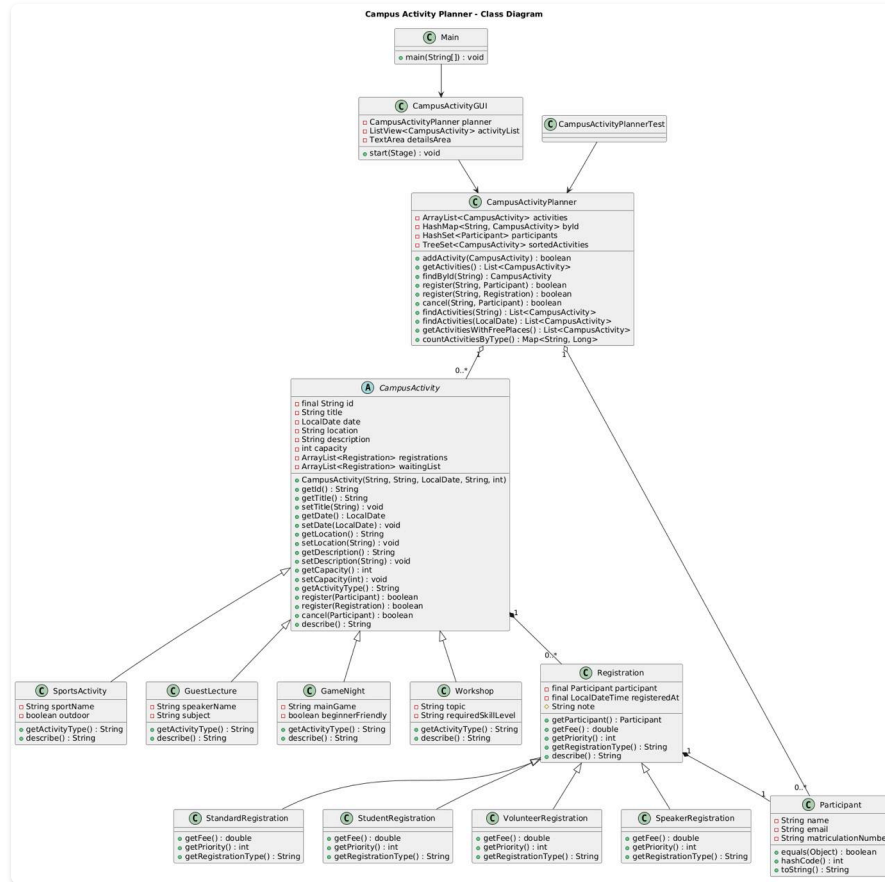
- Header includes the required image
- Left side: sorted activity list
- Center: details, confirmed participants and waiting list
- Right side: tabs for hosting and joining activities
- Buttons: `Add Activity` , `Register` , `Show free places` , `Show all`

```
ImageView poster = new ImageView(loadCampusImage());
poster.setFitWidth(230);
poster.setFitHeight(110);

Button registerButton = new Button("Register");
registerButton.setOnAction(event ->
    registerSelectedParticipant());

Button freeOnlyButton = new Button("Show free places");
freeOnlyButton.setOnAction(event -> activityList.setItems(
    FXCollections.observableArrayList(
        planner.getActivitiesWithFreePlaces())));
```

UML Class Diagram

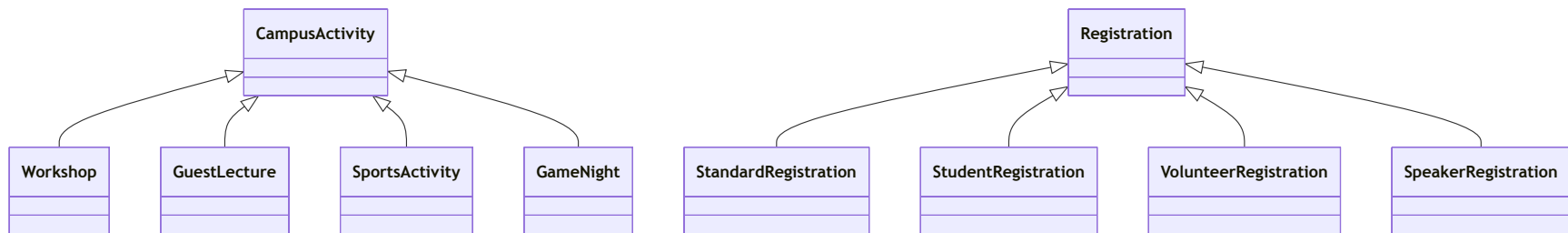


Classes We Created and Why

Class	Purpose
CampusActivity	Abstract base class for shared activity data and behavior
Workshop , GuestLecture , SportsActivity , GameNight	Concrete activity types with specific fields
Participant	Stores person data and duplicate identity logic
Registration	Connects a participant to an activity
Student , Standard , Volunteer , SpeakerRegistration	Different fees and waiting-list priorities

Inheritance

Two inheritance hierarchies are used.



Why:

- Activity subclasses share id, title, date, location, capacity and registration behavior
- Registration subclasses share participant and timestamp, but customize fee and priority

Polymorphism and Overriding

CampusActivity references can call subclass behavior.

```
public abstract class CampusActivity {  
    public abstract String getActivityType();  
    public String describe() { ... }  
}  
  
public class Workshop extends CampusActivity {  
    @Override  
    public String getActivityType() {  
        return "Workshop";  
    }  
}
```

The same pattern is used in Registration :

- getFee() returns 5.0 for students, 0.0 for volunteers/speakers
- getPriority() gives waiting-list priority to speakers and volunteers
- getRegistrationType() changes the display text

Composition

The design also uses HAS-A relationships.

```
public abstract class CampusActivity {  
    private final ArrayList<Registration> registrations;  
    private final ArrayList<Registration> waitingList;  
}  
  
public class Registration {  
    private final Participant participant;  
}
```

Why:

- A campus activity has many registrations
- A registration has one participant
- This keeps activity logic separate from participant identity data

Collections

`CampusActivityPlanner` uses multiple collections because each one has a different job.

Collection	Used for	Why
<code>ArrayList<CampusActivity></code>	Main activity list	Keeps insertion order and is easy to display
<code>HashMap<String, CampusActivity></code>	Activity lookup by id	Fast access with <code>findById()</code>
<code>HashSet<Participant></code>	Unique participants	Prevents duplicate participant objects
<code>TreeSet<CampusActivity></code>	Activities sorted by date	Uses <code>Comparable</code> automatically

HashMap, HashSet, Comparator

```
private final HashMap<String, CampusActivity> byId;  
private final HashSet<Participant> participants;  
private final TreeSet<CampusActivity> sortedActivities;  
  
public CampusActivity findById(String id) {  
    return byId.get(id.trim());  
}  
  
participants.add(registration.getParticipant());  
  
return activities.stream()  
    .sorted(Comparator.comparingInt(CampusActivity::getCapacity)  
        .thenComparing(CampusActivity::getTitle))  
    .collect(Collectors.toList());
```

The `Comparator` sorts by capacity first and title second.

Lambdas and Streams

Streams are used for searching, filtering, grouping and mapping.

```
public List<CampusActivity> getActivitiesWithFreePlaces() {  
    return activities.stream()  
        .filter(CampusActivity::hasFreePlaces)  
        .collect(Collectors.toList());  
}  
  
public Map<String, Long> countActivitiesByType() {  
    return activities.stream()  
        .collect(Collectors.groupingBy(  
            CampusActivity::getActivityType, Collectors.counting()));  
}  
  
public List<String> getDisplayLines() {  
    return activities.stream()  
        .map(activity -> activity.getId() + " | " + activity.describe())  
        .collect(Collectors.toList());  
}
```

equals() and hashCode()

Participant defines equality by normalized email address.

```
public void setEmail(String email) {
    if (email == null || !email.contains("@")) {
        throw new IllegalArgumentException("E-mail must contain @.");
    }
    this.email = normalizeEmail(email);
}

private String normalizeEmail(String value) {
    return value.trim().toLowerCase();
}
```

```
@Override
public boolean equals(Object other) {
    if (this == other) {
        return true;
    }
    if (!(other instanceof Participant)) {
        return false;
    }
    Participant that = (Participant) other;
    return Objects.equals(email, that.email);
}
```

JUnit Test

CampusActivityPlannerTest covers three important behaviors.

```
@Test
void fullActivityUsesWaitingListAndPromotesAfterCancel() {
    assertTrue(planner.register("T01",
        new StudentRegistration(first)));
    assertTrue(planner.register("T01",
        new VolunteerRegistration(second)));
    assertEquals(1, workshop.getWaitingList().size());
}

@Test
void duplicateParticipantCannotRegisterTwiceForSameActivity() {
    Participant duplicate = new Participant("Xue Wang", "XUE@example.com");
    assertFalse(planner.register("W01", duplicate));
}
```

Also tested:

- waiting-list promotion after cancellation
- overloaded findActivities(String) and findActivities(LocalDate)

Error Found and Debugged

Problem found while checking duplicate participant behavior:

- README says participants should be equal by email
- Test creates `xue@example.com` and `XUE@example.com`
- Email is normalized to lowercase
- Old equality logic compared matriculation number instead
- Old `hashCode()` used both email and matriculation number

Fix:

```
return Objects.equals(email, that.email);
```

```
return Objects.hash(email);
```

Result: `HashSet` and duplicate registration checks now use the same identity rule.

Demo Flow

1. Start `CampusActivityGUI`
2. Select `JavaFX Mini Workshop`
3. Show the campus image in the header
4. Register a participant with a name, email and registration type
5. Fill a small-capacity activity to show the waiting list
6. Use `Show free places` to demonstrate stream filtering
7. Try registering the same email twice to show duplicate prevention

What We Would Improve

- Save activities and registrations to a file or database
- Add stronger input validation for email and dates
- Add editing and deleting activities in the GUI
- Add search/filter controls directly in the GUI
- Add more tests for invalid inputs and registration priorities
- Improve the GUI layout and styling for smaller screens

Thank You

Questions?